Circular Campus

Phase 6 Work Distribution Report: Implementation Environment, Selected Features and Owners

Stage	Group	Course	Date
Phase 6 (plan)	Group 6	COSC 336	26 October 2026

Saeed Alkaabi 100063100 · Ali Alawadhi 100063186 · Humaid Al Ali 100062914 · Abdullah Alblooshi 100061739

This report answers the Phase 6 lab task: decide the implementation environment and the features we will implement, and say who implements what. Coding starts after the Phase 4 and 5 Design Document is submitted. The Phase 6 prototype is due in the labs of the week starting 22 November 2026.

1 Summary

We will build one Django web application, following the architecture and class diagrams of our Phase 4 and 5 Design Document. The lab asks for at least the first six core features (C1 to C6) and at least two AI features. Because the group now has four members, we plan all nine core features and three AI features:

- Core: C1 to C6 are required and come first; C7, C8 and C9 follow in the same phase.
- AI: #1 Classification and tagging, #2 Matching and ranking, and #5 Generated report. These are also the three AI functions the project brief lists for Phase 6.
- AI #3 Sustainable-action advice and #4 Assistant are not in this draft. They are kept for Phase 8 if time allows.

Abdullah Alblooshi joined the group on 26 October 2026, after the design sections were already split. He takes the request side of the system (C3, C4, C8, C9), which is self-contained and is fully described by our mockups and sequence diagrams, so he can start without having written a design section.

2 Implementation Environment

The environment follows the decisions already recorded in Phase 2 and in Section 3 of the Design Document (AD-1 to AD-6), so nothing in the design has to change.

Part	Choice	Why (design link)
Language	Python 3.12	One language for the web app and the AI code (Phase 2)
Web framework	Django 5.2 (LTS)	Built-in login, password hashing, forms, ORM and admin; one modular monolith (AD-1)
User interface	Django templates, one shared stylesheet, a little JavaScript	Server-rendered pages validated on the server; layout copied from our eight mockups (AD-5, §7)
Database	PostgreSQL 16 with the pgvector extension	Assets and their vectors in one database and one backup (AD-4, §9)
Embeddings	all-MiniLM-L6-v2 through the sentence-transformers library	Free, runs offline on the demo PC, used for classification of text and for matching (AD-3)
Text generation	OpenAI API through the official openai Python library, behind our AI gateway	Report summary and photo/PDF classification; key kept in .env, spending limit within AED 150 (AD-2, §3.3)

Background work	A Django management command that reads a jobs table every 30 seconds	Report drafts and vector updates run without a separate queue server (AD-6)
Version control	Git with one GitHub repository; feature branches and pull requests	Brief requires regular commits with clear individual authorship
Editor and tools	VS Code, pgAdmin, Django test runner	Same setup on all four laptops
Demo machine	One Windows laptop running Django, PostgreSQL and the worker locally	Deployment diagram, Figure 28

Every member uses the same requirements.txt and a .env.example file (no real keys in GitHub). Only synthetic asset records are used, so no real staff data is sent to the OpenAI API.

3 Selected Features

3.1 Core features

ID	Feature	Selected	Owner	What we will show
C1	Login and roles	Yes (required)	Ali	Requester, Custodian, Approver and Administrator roles; every view checks the access matrix on the server
C2	Departments and assets	Yes (required)	Saeed	Register an asset with category, condition, department and status
C3	Publish and search	Yes (required)	Abdullah	Publish surplus items; filter by keyword, category, condition and department
C4	Request and track	Yes (required)	Abdullah	Submit a request and see the status of my requests
C5	Approve and transfer	Yes (required)	Humaid	Approval step with no self-approval; receiver confirms; owner and department change; the last unit cannot go to two people
C6	States and history	Yes (required)	Ali	Only the moves in the asset state diagram are allowed; each asset shows who did what and when
C7	Sustainability dashboard	Yes	Saeed	Items reused, cost avoided (AED), waste diverted (kg) and kg CO2e, each with its formula and source (§12)
C8	Notifications	Yes	Abdullah	In-app messages when a request is made, approved, rejected or received
C9	Basic report	Yes	Abdullah	Table of requests and transfers with CSV export

3.2 AI features

#	AI feature	Selected	Owner	How we build it
1	Classification and tagging	Yes	Humaid	ClassificationService compares the description with category examples using MiniLM; returns category, condition, tags and a confidence. The custodian accepts or edits (approval point in §5).
2	Matching and ranking	Yes	Ali	MatchingService finds similar available assets with pgvector; RankingService applies the §11 weighted score and returns the top 5 with a one-line reason. The requester accepts or rejects.
5	Generated report	Yes	Saeed	The database calculates the dashboard numbers; SummaryService asks the OpenAI API for a short draft. Any number that differs from the query blocks publishing. The admin edits before publishing.
3	Sustainable-action advice	Later (Phase 8)	-	Kept in the design (ActionRecommender); not needed for the draft
4	Assistant (chat)	Later (Phase 8)	-	Kept in the design (AssistantService); needs all other data in place first

Why these three: they cover the start, middle and end of the cycle (Registered, Requested, Dashboard and report), they are the three AI functions named in the project brief for Phase 6, and two of them run locally, so most of the demo still works if the internet or the API key fails. Every AI result is a suggestion that a person confirms, and every AI call is written to the AI record (§13).

4 Work Distribution

Each member owns whole features, from the database model to the screen, so each person's commits show a complete piece of the system. Owners were matched to the design sections they wrote in Phase 4 and 5.

Member	Features	Django app and classes	Screens	Design sections used
Saeed Alkaabi 100063100	Project setup and data model; C2 Departments and assets; C7 Sustainability dashboard; AI #5 Generated report	Repository, settings, all models and migrations, synthetic seed data; assets app: Department, Category, Asset, ITAsset, LabEquipment, FurnitureAsset, Attachment; reports app: dashboard queries, SummaryService	Base page layout, M1 Asset registration, M6 Dashboard, summary part of M7	§7 Mockups, §8.2 Class diagram, §9 ERD and data dictionary, §12 Sustainability calculations
Ali Alawadhi 100063186	C1 Login and roles; C6 States and history; AI #2 Matching and ranking	accounts app: User, roles, can(user, action, record) check used by every view; AssetStatus transitions; AuditService.log(); ai app: MatchingService, RankingService, AI record	Login, user admin, asset history page, M3 Ranked matches	§6 Security (roles, access matrix, audit log), §8.5 State diagrams, §11 Recommendation score, §13 Monitoring
Humaid Al Ali 100062914	AI gateway; AI #1 Classification and tagging; C5 Approve and transfer	ai app: AIGateway, AIService, Suggestion, TextGenerator, OpenAIGenerator, LocalGenerator, MiniLMEmbedding, ClassificationService; transfers app: Reservation, Approval, Transfer	Suggestion panel on M1, M4 Approval, transfer release and receipt confirmation	§3.3 OpenAI API use, §4 AI services, §5 Approval points, §8.3 to 8.4 Sequence and activity, §10 Interface contracts
Abdullah Alblooshi 100061739	C3 Publish and search; C4 Request and track; C8 Notifications; C9 Basic report	exchange app: ResourceRequest, RequestSpecification, search and filter views; notifications app: notify(user, message, record) and inbox; reports app: requests/transfers table and CSV export	Catalogue and search, M2 New request, My requests, Inbox, table part of M7	§7 Mockups M2 and M7, §8.1 Use cases UC-08 to UC-12 and UC-29, §8.3 Request sequence, §9 data dictionary

4.1 Shared pieces and who provides them

Some code is used by everyone. The owner writes it first and the others call it instead of writing their own:

Shared piece	Provided by	Used by	Needed by
Models, migrations and seed data	Saeed	Everyone	End of week 1
Base page layout and stylesheet	Saeed	Everyone	End of week 1
can() permission check and role decorator	Ali	Every view	End of week 1
AuditService.log() and state transition check	Ali	C2, C4, C5	Middle of week 2
notify(user, message, record)	Abdullah	C4, C5	Middle of week 2
AIGateway and MiniLMEmbedding.embed()	Humaid	AI #1, #2, #5	End of week 2

5 Code Structure

The code is split into classes taken from our class diagrams (Figures 17 and 18), and each module of the architecture view (Figure 4) becomes one Django app. Views only read the form and call a service class; the rules (no self-approval, allowed state moves, last unit) live in the model or service classes, not in main or in the templates.

Django app	Architecture module	Main classes	Owner
accounts	Accounts & Administration	User, Role, AuditService	Ali
assets	Assets & Catalogue	Department, Category, Asset and its three subclasses, Attachment	Saeed
exchange	Search & Requests	ResourceRequest, RequestSpecification, MatchRecommendation	Abdullah (Ali for MatchRecommendation)
transfers	Reservations, Approvals & Transfers	Reservation, Approval, Transfer	Humaid
reports	Reporting & Sustainability	Dashboard queries, CSV export, SummaryService	Saeed and Abdullah
notifications	Notifications	Notification, notify()	Abdullah
ai	AI gateway	AIGateway, AIService, Suggestion, TextGenerator and adapters, the three AI services	Humaid and Ali

6 How We Work Together

1. **One repository.** Saeed creates the GitHub repository and adds all four members, the instructor and the lab engineer.
2. **One branch per feature.** For example feature/c5-transfer. Nobody commits straight to main.
3. **Small, regular commits.** Each member commits under their own GitHub account at least a few times a week, so the history shows who built what.
4. **Pull request review.** Every pull request is reviewed by one other member before it is merged into main.
5. **Tests with each feature.** Each owner adds Django tests for their own rules (for example, an approver cannot approve their own request). These are reused in Phase 7.
6. **Weekly check.** A short meeting after each lab to merge, run the full demo on the demo laptop and move work if someone is behind.

7 Timeline

Week	Dates	Goal	Done when
0	26 Oct to 1 Nov	Submit Phase 4 and 5; set up repository; Abdullah reads the design	Everyone can run the empty project locally
1	2 to 8 Nov	Models and migrations, seed data, base layout, login and roles (C1)	Four test users with different roles can sign in

2	9 to 15 Nov	C2 to C6 and C8; AI gateway	An asset goes from registration to a confirmed transfer, with history
3	16 to 21 Nov	AI #1, #2 and #5; C7 dashboard; C9 report and CSV; fixes	Full demo runs on the demo laptop from a fresh database
Demo	Week of 22 Nov	Phase 6 lab demonstration	Each member presents their own features

8 Risks

Risk	What we do
A new member joining late	Abdullah's features are built from finished mockups and sequence diagrams; Saeed pairs with him in week 0 to set up the project
OpenAI API key, cost or internet fails during the demo	AI #1 and #2 run locally; the report shows numbers without a summary; LocalGenerator can be switched on by one setting (AD-2)
Shared code is late and blocks others	Shared pieces in §4.1 have fixed dates and are merged first
Merge conflicts in models	Only Saeed changes migrations; others ask him for model changes